

DreamLog

Compression Enables Generalization in Logic Programming

Alexander Towell

10.5281/zenodo.19490027 (preprint)

Southern Illinois University Edwardsville

April 2026

Problem. Knowledge bases grow but do not learn. You can add 300 facts about a family tree, and the system still cannot tell you whether a new person is someone's father. The facts accumulate, understanding does not.

Claim. Compression *is* learning (Solomonoff's idea). The shortest KB consistent with the data generalizes best.

System. DreamLog, a Prolog variant with wake-sleep cycles inspired by DreamCoder.

- **Wake:** answer queries against the KB, and if a predicate is undefined, optionally ask an LLM to propose rules for it.
- **Sleep:** run 8 compression operations (7 symbolic, 1 LLM-assisted), each verified against positive and negative test queries with atomic rollback.

Every compression is behaviorally equivalent to the original KB on the verification suite. Nothing gets "worse."

What “compression discovers concepts” looks like

Pattern: compression via exceptions. Pick a default that covers most cases, list the exceptions. You gain compression whenever the exceptions are fewer than the non-exceptions.

From the crafting experiment, symbolic only (no LLM).

Input: seven ground facts, one per master artisan.

```
(master_artisan kael)      (master_artisan sera)
(master_artisan yara)      (master_artisan dax)
(master_artisan mira)      (master_artisan orin)
(master_artisan zara)
```

After one sleep cycle:

```
(master_artisan X) :- (artisan X),
                      (not (exception_master_artisan_artisan X)).
(exception_master_artisan_artisan thom).
(exception_master_artisan_artisan fen).
```

Seven facts become three clauses: “every artisan is a master unless listed.” The system invents a fresh predicate named `exception_{head}_{guard}` to hold the two exceptions.

At test time, adding `(artisan venn)` makes `(master_artisan venn)` derivable. Nobody wrote that fact; the system classified a new entity using a concept it discovered from structure alone.

The result I care most about

The worked example above is one rule from the crafting experiment. Aggregate over 33 test checks (19 positive, 14 negative), 5 runs per condition:

Condition	Recall	Precision	Rules
No dream	0%	—	0
Raw LLM prompting	0%	—	0
Symbolic only	53%	59%	5
Full pipeline	$64 \pm 2\%$	64%	20

The raw LLM scores 0% because the terms are invented. Symbolic compression alone hits 53% without any LLM involvement, which means compression discovers real concepts from data structure, not from prior knowledge. The LLM adds another 11 points on top.

This is the cleanest evidence I have for the thesis.

The paper needs work before it's venue-ready: a head-to-head Popper or Metagol baseline, figures, and likely a larger-scale run.

What coauthoring might look like. Where I think you could contribute:

- Benchmark and metric design: what tasks should DreamLog be evaluated on, what counts as success, how to scope a fair comparison with Popper. Your ransomware work is a good model for turning fuzzy claims into concrete thresholds.
- Venue strategy; a senior coauthor materially changes what's reachable
- Framing with respect to my dissertation: is this a chapter or a side project?

What I'd do. Everything else: implementation, statistical analysis, writing, figures, running the benchmark once we've scoped it, revisions.